Set 22: Aggregate Statistics with Pandas

Skill 22.1: Describe the purpose of an aggregate statistic

Skill 22.1 Concepts

An *aggregate* statistic is a way of creating a single number that describes a group of numbers. Common aggregate statistics include mean, median, and standard deviation.

More specifically, aggregate statistics:

- **Reduce complexity** by condensing many data points into a single number or a small set of numbers
- **Describe overall behavior** of a dataset (e.g., typical value, spread, totals)
- **Enable comparisons** between groups, time periods, or categories
- **Support decision-making** by highlighting trends rather than individual noise

Skill 22.1 Exercise 1

Skill 22.2: Calculate column statistics

Skill 22.2 Concepts

Aggregate functions summarize many data points (i.e., a column of a dataframe) into a smaller set of values.
Some examples of aggregate functions built into Pandas are illustrated below,

Code

```
display(customers.head())
```

Output

	id	first_name	last_name	age	state
0	41874	Kyle	Peck	30	SC
1	31349	Elizabeth	Velazquez	21	NC
2	43416	Keith	Saunders	36	NJ
3	56054	Ryan	Sweeney	40	ID
4	77402	Donna	Blankenship	48	NM

Explanation

Prints the first 5 rows of of the customers dataframe

Code

```
print(customers.age.median())
```

Output

33.0

Explanation

Prints the median of the values in the age column

Code	
`print(customers.state.nunique())`	
Output	**Explanation**
46	Prints the number of unique values in the state column
Code	
`print(customers.state.unique())`	
Output	**Explanation**
['ME' 'DE' 'KS' 'AZ' 'GA' 'NY' 'WV' 'NH' 'WI' 'MI' 'ND' 'IN' 'NC' 'AL' 'PA' 'MA' 'MO' 'AK' 'MN' 'SD' 'HI' 'WA' 'MD' 'FL' 'CT' 'WY' 'IA' 'OR' 'KY' 'CA' 'IL' 'NJ' 'NV' 'NM' 'OK' 'TX' 'MT' 'CO' 'UT' 'LA' 'RI' 'ID' 'SC' 'MS' 'OH' 'VA']	Prints a list of the unique values in the state column

The following table summarizes some of the more common commands,

Method	Description
mean	Average of all values in column
std	Standard deviation
median	Median
max	Maximum value in column
min	Minimum value in column
count	Number of values in column
nunique	Number of unique values in column
unique	List of unique values in column
value_counts	counts how many times each unique value appears

Skill 22.2 Exercise 1

Skill 22.3: Apply pandas functions to compute aggregate statistics

Skill 22.3 Concepts

When we have a bunch of data, we often want to calculate aggregate statistics (mean, standard deviation, median, percentiles, etc.) over certain subsets of the data.

Let's revisit our customer data which contains some new columns.

	id	first_name	last_name	email	shoe_type	shoe_material	shoe_color	price	age	state
0	41874	Kyle	Peck	KylePeck71@gmail.com	ballet flats	faux-leather	black	385.0	31	NH
1	31349	Elizabeth	Velazquez	EVelazquez1971@gmail.com	boots	fabric	brown	388.0	22	AR
2	43416	Keith	Saunders	KS4047@gmail.com	sandals	leather	navy	346.0	36	FL
3	56054	Ryan	Sweeney	RyanSweeney14@outlook.com	sandals	fabric	brown	344.0	21	KY
4	77402	Donna	Blankenship	DB3807@gmail.com	stilettos	fabric	brown	289.0	33	UT

Below illustrates how we could calculate the average cost of each type of shoe

Code

```
print(customers.groupby('shoe_type').price.mean())
```

Output

```
shoe_type
ballet flats    256.000000
boots           267.684211
clogs           310.625000
sandals         274.125000
stilettos       360.928571
wedges          277.777778
Name: price, dtype: float64
```

Below is an explanation of each part of the code above,

1. customers

Your pandas DataFrame with customer info (including shoe_type and price).

2. .groupby('shoe_type')

This groups the rows by the type of shoe.
So all "sneakers" together, all "boots" together, all "sandals" together, etc.
At this stage, pandas is just organizing the data—no math yet.

3. .price

From each shoe_type group, this selects the price column.
You're telling pandas:
"Within each shoe type, look at the prices."

4. .mean()

This calculates the average (mean) price *for each shoe type*.
So pandas:

- Takes all prices for sneakers → computes their average
- Takes all prices for boots → computes their average
- …and so on

The syntax above can be summarized as follows,

```
df.groupby('CATEGORY_COLUMN').NUMERIC_COLUMN.AGGREGATION()
```

After using *groupby*, we often need to clean our resulting data.

The *groupby* function creates a new Series, not a DataFrame. In the previous example, the indices of the Series were different values of shoe_type, and the name property was price.

Usually, we'd prefer that those indices were actually a column. In order to get that, we can use *reset_index().* This will transform our Series into a DataFrame and move the indices into their own column.

Generally, you'll always see a *groupby* statement followed by *reset_index*. The example below illustrates the effect of *reset_index().*

Code

```
display(customers.groupby('shoe_type').price.mean())
```

Output	**Explanation**
shoe_type ballet flats 256.000000 boots 267.684211 clogs 310.625000 sandals 274.125000 stilettos 360.928571 wedges 277.777778 Name: price, dtype: float64	Returns a series with no index column

Code

```
display(customers.groupby('shoe_type').price.mean().reset_index())
```

Output	**Explanation**
	*reset_index()* transforms the series into a dataframe and moves the indices to their own column

	shoe_type	price
0	ballet flats	256.000000
1	boots	267.684211
2	clogs	310.625000
3	sandals	274.125000
4	stilettos	360.928571
5	wedges	277.777778

Skill 22.3 Exercise 1

Skill 22.4: Apply lambda functions to calculate aggregate statistics

Skill 22.4 Concepts

Sometimes, the operation that you want to perform is more complicated than mean or count. In those cases, you can use the apply method and lambda functions, just like we did for individual column operations. Note that the input to our lambda function will always be a list of values.

A great example of this is calculating percentiles. Suppose we have a DataFrame of employee information called df that has the following columns,

- id: the employee's id number
- name: the employee's name
- wage: the employee's hourly wage
- category: the type of work that the employee does

Our data might look something like this,

id	name	wage	category
10131	Sarah Carney	39	product
14189	Heather Carey	17	design
15004	Gary Mercado	33	marketing
11204	Cora Copaz	27	design

If we want to calculate the 75th percentile (i.e., the point at which 75% of employees have a lower wage and 25% have a higher wage) for each category, we can use the following combination of apply and a lambda function,

Code

```
high_earners = df.groupby('category').wage
    .apply(lambda x: np.percentile(x, 75))
    .reset_index()
```

Output

	category	wage
0	design	23
1	marketing	35
2	product	48
…		

Skill 22.4 Exercise 1

Skill 22.5: Compute aggregate statistics using multiple columns

Skill 22.5 Concepts

Sometimes, we want to group by more than one column. We can easily do this by passing a list of column names into the groupby method.

Imagine that we run a chain of stores and have data about the number of sales at different locations on different days,

Location	Date	Day of Week	Total Sales
West Village	February 1	W	400
West Village	February 2	Th	450
Chelsea	February 1	W	375
Chelsea	February 2	Th	390

We suspect that sales are different at different locations on different days of the week. To test this hypothesis, we could calculate the average sales for each store on each day of the week across multiple months. The code would look like this,

Code

```
df.groupby(['Location', 'Day of Week'])['Total Sales'].mean().reset_index()
```

Output

Location	Day of Week	Total Sales
Chelsea	M	402.50
Chelsea	Tu	422.75
Chelsea	W	452.00
…		
West Village	M	390
West Village	Tu	400
…		

Skill 22.5 Exercise 1

Skill 22.6: Create a pivot table with pandas

Skill 22.6 Concepts

When we perform a groupby across multiple columns, we often want to change how our data is stored. For instance, recall the example where we are running a chain of stores and have data about the number of sales at different locations on different days,

Location	Date	Day of Week	Total Sales
West Village	February 1	W	400
West Village	February 2	Th	450
Chelsea	February 1	W	375
Chelsea	February 2	Th	390

We suspected that there might be different sales on different days of the week at different stores, so we performed a `groupby` across two different columns (`Location` and `Day of Week`). This gave us results that looked like this,

Location	Day of Week	Total Sales
Chelsea	M	300
Chelsea	Tu	310
Chelsea	W	375
Chelsea	Th	390
…		
West Village	Th	450
West Village	F	390
West Village	Sa	250
…		

To test our hypothesis, it would be more useful if the table was formatted like this,

Location	M	Tu	W	Th	F	Sa	Su
Chelsea	300	310	375	390	300	150	175
West Village	300	310	400	450	390	250	200

Reorganizing a table in this way is called **pivoting**. The new table is called a **pivot table**.

In pandas, the command for pivot is,

```
df.pivot(columns='ColumnToPivot',
         index='ColumnToBeRows',
         values='ColumnToBeValues')
```

Below is an example using our sales data,

Code

```
# First use the groupby statement:
unpivoted = df.groupby(['Location', 'Day of Week'])['Total Sales'].mean().reset_index()
# Now pivot the table
pivoted = unpivoted.pivot(
    columns='Day of Week',
    index='Location',
    values='Total Sales')
```

Output

Location	M	Tu	W	Th	F	Sa	Su
Chelsea	300	310	375	390	300	150	175
West Village	300	310	400	450	390	250	200

Just like with groupby, the output of a pivot command is a new DataFrame, but the indexing tends to be "weird", so we usually follow up with *reset_index().*

Skill 22.6 Exercise 1